Pacific Firestorm - Complete Codex Godot Blueprint

All prompts, phases, architecture, data, testing, and visual direction

Part 1 - Project Overview

Purpose of This Kit

This kit is a complete Codex-agent handoff for building Pacific Firestorm: an original Godot 4 arcade aerial shooter inspired by the structure of classic vertical shooters, but with ultra-realistic top-down Pacific air-combat visuals.

Core rule: arcade gameplay, realistic visuals, original identity.

Game Goal

Build a vertical scrolling aerial shooter with simple, responsive arcade controls, enemy waves, power-ups, bosses, score chasing, and modern realistic presentation: aircraft, ocean, islands, smoke, fire, tracer rounds, explosions, weather, ships, and aircraft carriers.

Legal and creative boundary

- Do not use the 1942 name, Capcom branding, original sprites, sound effects, music, UI, enemy waves, or level layouts.
- Use the broad genre structure only: move, dodge, shoot, survive waves, collect power-ups, defeat bosses, chase score.
- Create original aircraft designs, original UI, original levels, original enemy wave timing, and original audio.

Recommended Technical Direction

Use a hybrid 3D/2D approach: Godot 4 with GDScript, a 3D world viewed by an orthographic Camera3D, and arcade gameplay locked to the X/Z plane.

Area	Recommendation
Engine	Godot 4.x
Language	GDScript first, C# only later if required
Camera	Camera3D, orthographic, top-down or slight tilt
Gameplay plane	X/Z plane with simplified hitboxes
Collision	Area3D and CollisionShape3D, fair arcade hitboxes
Data	JSON-driven enemies, waves, stages, weapons, power-ups

MVP Definition

The first real milestone is one complete playable stage, not a giant campaign. The MVP must prove that the game is fun before visual polish and extra content are expanded.

- Title screen and start game flow.
- Player aircraft with 8-direction arcade movement.
- Shooting, bullets, collisions, score, lives, HUD.
- Enemy fighters, enemy bullets, bomber, power-up, first boss.
- Scrolling ocean, simple clouds, basic explosions, game over, mission complete.

Build Strategy

Use Codex in phases. Give one phase prompt, run the Godot project, copy errors back to Codex, request minimum fixes, then continue. Avoid massive untested feature dumps.

```
1. Foundation
2. Main scene
3. Player movement
4. Shooting
5. Enemies
6. Collision/score
7. Lives/HUD
8. Wave spawner
9. Environment
10. Power-ups
11. Boss
12. Menus
13. Effects/audio
14. Stage 1
15. Visual realism
16. Performance/export
```

Repository Structure

```
PacificFirestorm/
  project.godot

  scenes/
    main/Main.tscn
    player/Player.tscn
    enemies/EnemyFighter.tscn
    enemies/EnemyBomber.tscn
    enemies/BossBomber.tscn
    bullets/PlayerBullet.tscn
    bullets/EnemyBullet.tscn
    bullets/Rocket.tscn
    bullets/FlakBurst.tscn
    powerups/Powerup.tscn
    effects/Explosion.tscn
    effects/SmokeTrail.tscn
    effects/WaterSplash.tscn
    effects/HitSpark.tscn
    environment/Ocean.tscn
    environment/IslandChunk.tscn
    environment/CloudLayer.tscn
    environment/Ship.tscn
    environment/Carrier.tscn
    ui/HUD.tscn
    ui/TitleScreen.tscn
    ui/PauseMenu.tscn
    ui/GameOverScreen.tscn
    ui/MissionCompleteScreen.tscn

  scripts/
    main/main.gd
    main/game_state.gd
    main/stage_manager.gd
    player/player.gd
    player/player_weapon.gd
    player/player_damage.gd
    enemies/enemy_base.gd
```

```
  enemies/enemy_spawner.gd
  enemies/movement_patterns.gd
  enemies/boss_controller.gd
  bullets/bullet_base.gd
  bullets/projectile_pool.gd
  powerups/powerup.gd
  powerups/powerup_spawner.gd
  effects/explosion.gd
  effects/effects_manager.gd
  effects/screen_shake.gd
  environment/ocean_scroller.gd
  environment/cloud_scroller.gd
  environment/environment_spawner.gd
  ui/hud.gd
  ui/title_screen.gd
  ui/pause_menu.gd
  ui/game_over_screen.gd
  ui/mission_complete_screen.gd
  systems/audio_manager.gd
  systems/input_manager.gd
  systems/save_manager.gd
  systems/config_manager.gd

data/
  stages/stage_list.json
  stages/stage_01.json
  waves/stage_01_waves.json
  enemies/enemy_types.json
  weapons/weapon_types.json
  powerups/powerup_types.json

assets/
  models/aircraft/
  models/ships/
  models/islands/
  textures/ocean/
  textures/aircraft/
  textures/explosions/
  textures/clouds/
  textures/ui/
  materials/water/
  materials/aircraft/
  materials/smoke/
  materials/tracer/
  audio/music/
  audio/sfx/
  fonts/
  placeholders/

docs/
  game_design.md
  art_direction.md
  technical_plan.md
  controls.md
  codex_tasks.md
  known_issues.md
  legal_notes.md
  export_guide.md
  testing_checklist.md
  release_checklist.md

.github/pull_request_template.md
AGENTS.md
README.md
```

Part 2 - Master Prompt

How to Use This PDF

Paste the master prompt into Codex at the start of the project. Then paste Phase 0. After each phase, test the project manually in Godot before continuing.

Master Codex Agent Prompt

```
You are the coding agent for a Godot 4 game project called Pacific Firestorm.

Project:
Pacific Firestorm is an original vertical-scrolling aerial shooter inspired by classic
    1942-style arcade gameplay, but with a modern ultra-realistic Pacific air-combat visual
    direction.

Important:
This is not a direct clone of 1942. Do not copy Capcom branding, name, sprites, music, sound
    effects, level layouts, enemy waves, UI, or any copyrighted assets. Build an original
    spiritual successor.

Engine:
Godot 4.x

Language:
GDScript

Visual style:
Top-down or slight-tilt orthographic 3D. Use realistic aircraft, ocean, islands, smoke, fire,
    clouds, tracer rounds, explosions, and ships. Use placeholders first.

Gameplay style:
Arcade. Responsive. Readable. Not a realistic flight simulator.

Core rule:
Arcade gameplay. Realistic visuals. Original identity.

Technical approach:
Use a 3D world with an orthographic Camera3D. Gameplay occurs on the X/Z plane. Use simple
    Area3D hitboxes. Use data-driven JSON files for enemies, waves, stages, weapons, and
    power-ups. Use object pooling for bullets and effects once the basic loop works. Keep the
    project runnable after every phase.

Required systems:
- Main scene
- Player aircraft
- Player movement
- Player weapon system
- Player bullets
- Enemy base class
- Enemy movement patterns
- Enemy spawner
- Enemy bullets
- Bullet collision
- Player damage/lives
- Score system
- HUD
- Stage manager
- Game states
- Title screen
- Pause menu
- Game over screen
- Mission complete screen
- Power-ups
- Boss system
- Effects manager
- Audio manager
- Scrolling ocean/environment
- Export documentation

Development rules:
- Implement in phases.
- Do not over-engineer.
- Do not build all content before the core gameplay works.
```

```
- Do not silently remove existing functionality.
- Keep scripts small and readable.
- Document unfinished items in docs/known_issues.md.
- Update README and docs when behaviour changes.
- Prefer simple working code over clever architecture.
- Use placeholder assets until final assets are provided.
- Use fair arcade hitboxes, not exact model collision.
- Make bullets and hazards readable even with realistic graphics.

First task:
Create the repository structure, AGENTS.md, README.md, docs files, and basic Godot project
    skeleton. Do not implement full gameplay yet.
```

First Start Prompt

```
Start the Pacific Firestorm Godot project.

Use the full project rules below:
- Godot 4.x
- GDScript
- Original vertical scrolling aerial shooter
- Inspired by classic 1942-style arcade gameplay but not a direct clone
- Ultra-realistic top-down Pacific air-combat visual direction
- 3D orthographic gameplay presentation
- Arcade movement, not realistic flight simulation
- Data-driven stages/waves/enemies/weapons
- Placeholder assets first
- Keep project runnable after every phase

First task:
Create only the foundation:
1. AGENTS.md
2. README.md
3. docs/game_design.md
4. docs/art_direction.md
5. docs/technical_plan.md
6. docs/controls.md
7. docs/known_issues.md
8. docs/legal_notes.md
9. .github/pull_request_template.md
10. Godot folder structure
11. Minimal project.godot
12. scenes/main/Main.tscn
13. scripts/main/main.gd

Main.tscn should contain:
- root Node3D named Main
- WorldRoot Node3D
- Camera3D with orthographic projection
- DirectionalLight3D
- placeholder ocean Node3D
- PlayerSpawn Node3D
- EnemyRoot Node3D
- PlayerBulletRoot Node3D
- EnemyBulletRoot Node3D
- EffectsRoot Node3D
- UI placeholder CanvasLayer

Do not implement full gameplay yet.
Do not add copyrighted assets.
Use placeholders only.
Keep everything simple and readable.

Acceptance criteria:
- Project opens in Godot.
- Main.tscn runs.
- No script parse errors.
- README explains how to open the project.
- AGENTS.md contains the long-term rules for future Codex tasks.
```

AGENTS.md

```
# AGENTS.md - Pacific Firestorm

## Project Summary
```

```
Pacific Firestorm is an original Godot 4 vertical-scrolling aerial shooter inspired by classic
    arcade air-combat shooters, but with a modern ultra-realistic visual direction.

The game should feel like a simple, fast, readable arcade shooter. It should not become a
    realistic flight simulator.

## Core Design Rule

Arcade gameplay. Realistic visuals. Original identity.

## Legal / Creative Boundaries

Do not copy:
- Capcom branding
- 1942 name
- original 1942 sprites
- original 1942 sound effects
- original 1942 music
- exact 1942 enemy waves
- exact 1942 level layouts
- exact 1942 UI
- recognizable direct clones of copyrighted assets

Create original aircraft designs, enemy waves, UI, effects, levels, names, music, and sounds.

## Engine

Use Godot 4.x.

## Language

Use GDScript unless specifically instructed otherwise.

## Game Type

Top-down / slight-tilt orthographic 3D vertical shooter.

Gameplay is 2D arcade logic on the X/Z plane. Visuals use 3D models, lighting, particles, and
    shaders.

## Camera

Use Camera3D with orthographic projection. The camera looks downward at the battlefield. The
    gameplay plane is X/Z. Y is used for height/visual layering if needed.

## Code Style

- Keep scripts small and readable.
- Prefer composition over giant manager scripts.
- Keep gameplay logic separated from rendering/effects.
- Use signals for major events where appropriate.
- Use data files for stages, enemies, waves, weapons, and power-ups.
- Keep the project runnable after every phase.
- Do not add advanced features before the core loop works.
- Do not over-engineer.

## Required Core Systems

- Main scene
- Player aircraft
- Player movement
- Player shooting
- Player damage/lives
- Enemy base class
- Enemy spawner
- Enemy movement patterns
- Bullet system
- Collision detection
- Score system
- HUD
- Game states
- JSON wave loading
- Stage manager
- Effects manager
- Audio manager
- Menus
- Export documentation

## Gameplay Feel

The game should be responsive and arcade-like.
```

```
Do not implement realistic:
- lift
- drag
- stalls
- fuel
- limited ammunition
- complex aerodynamics
- long turning radius

The player should move instantly and precisely.

## Hitboxes

Use fair arcade-style hitboxes. The visual model may be large, but the actual player damage
    hitbox should be small and readable.

## Readability

Even with realistic graphics, bullets, enemy fire, power-ups, and hazards must remain visible.

Use:
- tracer glow
- clear silhouettes
- UI highlights
- readable contrast
- consistent bullet colours

## Performance Rules

Use object pooling for:
- player bullets
- enemy bullets
- explosions
- smoke trails
- hit sparks

Avoid:
- excessive real-time lights
- excessive transparent particles
- huge uncompressed textures
- high-poly models for small screen objects

## Data-Driven Design

Use JSON for:
- enemy types
- weapon types
- power-up types
- stages
- enemy waves

Do not hard-code whole stages in scripts.

## Development Discipline

For every task:
1. Make the smallest useful change.
2. Keep the project runnable.
3. Update documentation if behaviour changes.
4. Add clear notes to docs/known_issues.md if something is unfinished.
5. Do not silently remove existing functionality.

## Acceptance Criteria for Every Phase

A phase is complete only when:
- the project opens in Godot
- there are no script parse errors
- the main scene runs
- the implemented feature works in a basic way
- obvious errors are documented or fixed
```

Part 3 - Phased Build Prompts

Phase Prompt Index

Paste one phase at a time. Do not let Codex skip ahead unless the project runs and the acceptance criteria are met.

Phase 0 - Repo Setup

```
You are working on a new Godot 4 GDScript project called Pacific Firestorm.

Before implementing gameplay, set up the repository structure and project documentation.

Create:
- AGENTS.md
- README.md
- docs/game_design.md
- docs/art_direction.md
- docs/technical_plan.md
- docs/controls.md
- docs/codex_tasks.md
- docs/known_issues.md
- docs/legal_notes.md
- .github/pull_request_template.md

Create the folder structure:
- scenes/
- scripts/
- data/
- assets/
- docs/

Use the structure described in AGENTS.md.

Do not implement gameplay yet.
Do not add copyrighted assets.
Use clear markdown documentation.

Acceptance criteria:
- The repository has the expected folders.
- AGENTS.md gives coding and design rules.
- README.md explains how the project will be opened and run later.
- docs/legal_notes.md states that this is an original game inspired by classic arcade shooters,
    not a direct clone.
```

Phase 1 - Godot Skeleton

```
Implement Phase 1: Godot project skeleton.

Create or update:
- project.godot
- scenes/main/Main.tscn
- scripts/main/main.gd

Main.tscn should contain:
- root Node3D named Main
- WorldRoot Node3D
- Camera3D using orthographic projection
- DirectionalLight3D
- WorldEnvironment if appropriate
- PlaceholderOcean Node3D
- PlayerSpawn Node3D
- EnemyRoot Node3D
- PlayerBulletRoot Node3D
- EnemyBulletRoot Node3D
- EffectsRoot Node3D
- UI CanvasLayer placeholder if practical

main.gd should:
- initialize the scene
- print a startup message
- expose references to important child roots
```

```
- not implement advanced gameplay yet

Set up basic input actions in project.godot:
- move_left
- move_right
- move_up
- move_down
- fire
- bomb
- evade
- pause

Acceptance criteria:
- Godot project opens.
- Main.tscn can be set as the main scene.
- Running the scene shows a camera, light, and placeholder world.
- No script parse errors.
```

Phase 2 - Player Aircraft Placeholder

```
Implement Phase 2: player aircraft placeholder.

Create:
- scenes/player/Player.tscn
- scripts/player/player.gd

Player.tscn should use:
- CharacterBody3D or Area3D-based movement, whichever is cleaner for this arcade shooter
- a visible placeholder mesh, such as a simple low-poly plane shape or box-based aircraft
    silhouette
- a child Node3D named GunPoints
- at least two child Node3D gun points: LeftGun and RightGun
- an Area3D or CollisionShape3D for damage hitbox

player.gd should:
- move the player on the X/Z plane
- use input actions move_left, move_right, move_up, move_down
- keep movement arcade-responsive
- clamp the player inside configurable playfield bounds
- not implement realistic flight physics
- expose speed and bounds as exported variables

Update Main.tscn:
- instance Player.tscn at PlayerSpawn

Acceptance criteria:
- The player appears in Main.tscn.
- The player moves in 8 directions.
- The player cannot leave the playfield.
- Movement feels immediate and arcade-like.
- No shooting yet unless trivial hooks are needed.
```

Phase 3 - Player Shooting and Bullet System

```
Implement Phase 3: player shooting and bullet system.

Create:
- scenes/bullets/PlayerBullet.tscn
- scripts/bullets/bullet_base.gd
- scripts/player/player_weapon.gd if useful

PlayerBullet.tscn should:
- be an Area3D
- have a visible placeholder tracer/bullet mesh
- have a CollisionShape3D
- move forward/up-screen along the gameplay direction
- destroy or deactivate itself when offscreen

Implement:
- fire cooldown
- dual gun firing from LeftGun and RightGun
- configurable bullet speed
- configurable bullet damage
- a simple bullet lifetime/offscreen cleanup

Prefer simple working implementation first.
```

```
If object pooling is not implemented yet, document that pooling will be added later in
    docs/known_issues.md.

Acceptance criteria:
- Pressing fire shoots bullets.
- Bullets spawn from the player gun points.
- Bullets travel upward/forward.
- Bullets disappear when offscreen.
- No errors when firing repeatedly.
```

Phase 4 - Enemy Base and First Enemy

```
Implement Phase 4: enemy base and first enemy.

Create:
- scripts/enemies/enemy_base.gd
- scenes/enemies/EnemyFighter.tscn
- scripts/enemies/movement_patterns.gd

EnemyFighter.tscn should:
- be an Area3D or Node3D with Area3D collision
- have a visible placeholder aircraft mesh
- have a CollisionShape3D
- use enemy_base.gd

enemy_base.gd should support:
- health
- score_value
- movement_speed
- movement_pattern
- take_damage(amount)
- die()
- signal died(score_value)
- cleanup when offscreen

movement_patterns.gd should include at least:
- straight_down
- diagonal_left
- diagonal_right
- sine_wave

Update Main.tscn or main.gd temporarily to spawn one enemy for testing.

Acceptance criteria:
- At least one enemy appears.
- Enemy moves down the screen.
- Enemy cleans up when offscreen.
- Enemy has health and can receive damage through take_damage().
```

Phase 5 - Bullet/Enemy Collision and Score

```
Implement Phase 5: bullet-enemy collision and score.

Create:
- scripts/main/game_state.gd
- scripts/ui/hud.gd
- scenes/ui/HUD.tscn

GameState should track:
- score
- lives
- bombs
- current_stage
- game_over state

HUD should display:
- score
- lives
- bombs
- stage name

Connect bullet collision:
- PlayerBullet detects enemy Area3D.
- If collided object has take_damage(), call it.
- Bullet disappears or deactivates after hit.
- Enemy emits died(score_value).
```

```
- GameState adds score.
- HUD updates score.

Keep the implementation simple and robust.

Acceptance criteria:
- Player bullets damage enemies.
- Enemy dies when health reaches zero.
- Score increases when enemy dies.
- HUD updates score.
- No duplicate score from one enemy death.
```

Phase 6 - Player Damage and Lives

```
Implement Phase 6: player damage and lives.

Add to player:
- take_damage(amount)
- invulnerability window after hit
- visual hit flash placeholder
- signal player_hit
- signal player_destroyed

Update GameState:
- reduce lives when player is hit
- trigger game over when lives reach zero
- reset or respawn player after hit if lives remain

For now, allow collision with enemy aircraft to damage player.
Enemy bullets can be added later.

Acceptance criteria:
- Colliding with an enemy damages the player.
- Player briefly becomes invulnerable after damage.
- Lives decrease on HUD.
- Game over state triggers when lives reach zero.
- Player is not instantly killed multiple times by one collision.
```

Phase 7 - Enemy Spawner

```
Implement Phase 7: enemy spawner.

Create:
- scripts/enemies/enemy_spawner.gd
- data/waves/stage_01_waves.json
- data/enemies/enemy_types.json

EnemySpawner should:
- load wave data from JSON
- track stage time
- spawn enemies at defined times
- support enemy_type
- support spawn_x
- support spawn_z
- support movement_pattern
- support formation_id or group_id if simple

Use EnemyFighter.tscn as the first enemy type.

Example wave entry:
{
  "time": 1.0,
  "enemy_type": "fighter",
  "spawn_x": -3.0,
  "spawn_z": -16.0,
  "movement": "straight_down"
}

Acceptance criteria:
- Stage wave JSON is loaded.
- Enemies spawn according to time.
- At least 10 enemies spawn during a short test wave.
- Enemy movement pattern is applied.
- The system is data-driven, not hard-coded.
```

Phase 8 - Enemy Bullets

```
Implement Phase 8: enemy bullet system.

Create:
- scenes/bullets/EnemyBullet.tscn
- script support in bullet_base.gd or separate enemy_bullet.gd

Enemy bullets should:
- be visible and readable
- move downward or toward player depending on mode
- use Area3D collision
- damage player on contact
- clean up offscreen

Update EnemyFighter:
- optional fire_timer
- simple downward shooting
- configurable fire_interval
- configurable bullet_scene

Acceptance criteria:
- Enemies fire bullets.
- Enemy bullets are readable on screen.
- Enemy bullets damage the player.
- Enemy bullets clean up when offscreen.
- Bullet collision does not damage enemies incorrectly.
```

Phase 9 - Object Pooling

```
Implement Phase 9: object pooling for bullets and simple effects.

Create:
- scripts/bullets/projectile_pool.gd
- optional scripts/effects/effects_pool.gd

Pooling should support:
- preloading bullet scenes
- reusing inactive bullets
- returning bullets to pool when offscreen or on hit
- avoiding excessive instantiate/free during gameplay

Do not rewrite the entire project.
Make the smallest safe change to replace repeated bullet instantiation with pooling.

Acceptance criteria:
- Player bullets use pool.
- Enemy bullets use pool if practical.
- Existing gameplay remains unchanged.
- No broken collision behaviour.
- Docs explain how the projectile pool works.
```

Phase 10 - Scrolling Ocean and Environment

```
Implement Phase 10: scrolling ocean/environment system.

Create:
- scenes/environment/Ocean.tscn
- scripts/environment/ocean_scroller.gd
- scenes/environment/CloudLayer.tscn
- scripts/environment/cloud_scroller.gd
- scripts/environment/environment_spawner.gd

Ocean should:
- provide a placeholder water plane
- scroll visually downward to create flight motion
- support future water material/shader replacement

CloudLayer should:
- spawn simple semi-transparent cloud placeholders
- scroll at a different speed than ocean
- not block gameplay readability

EnvironmentSpawner should eventually support:
- islands
```

```
- ships
- smoke columns
- carriers
- wake trails

Acceptance criteria:
- Ocean/background scrolls continuously.
- Player appears to fly forward.
- Clouds or placeholders move at a different speed.
- Gameplay remains readable.
```

Phase 11 - Power-Ups

```
Implement Phase 11: power-up system.

Create:
- scenes/powerups/Powerup.tscn
- scripts/powerups/powerup.gd
- data/powerups/powerup_types.json

Power-up types:
- weapon_upgrade
- repair
- bomb
- score_bonus
- rapid_fire_temporary

Powerups should:
- move downward slowly
- have clear visual placeholder
- be collected by player
- apply effect through player or GameState
- clean up offscreen

Update enemies:
- allow optional drop_chance
- allow optional drop_type

Acceptance criteria:
- Enemies can drop power-ups.
- Player can collect power-ups.
- Weapon upgrade changes player firing pattern or bullet count.
- Repair increases lives or health according to current design.
- HUD updates if relevant.
```

Phase 12 - Weapon Upgrades

```
Implement Phase 12: weapon upgrade system.

Create or update:
- data/weapons/weapon_types.json
- scripts/player/player_weapon.gd

Weapon levels:
Level 1: dual machine guns
Level 2: wider dual guns + faster fire
Level 3: triple shot
Level 4: spread shot
Special: bomb/rocket attack

Rules:
- Keep bullets readable.
- Do not flood the screen so much that gameplay becomes unclear.
- Make weapon level configurable.
- Clamp max weapon level.
- Reset weapon level on death only if current game design says so.

Acceptance criteria:
- Collecting weapon_upgrade increases weapon level.
- Different weapon levels visibly change shot pattern.
- Weapon settings come from data or clearly structured code.
- Shooting remains stable and performant.
```

Phase 13 - Boss System

```
Implement Phase 13: boss system.

Create:
- scripts/enemies/boss_controller.gd
- scenes/enemies/BossBomber.tscn or BossCarrier.tscn

Boss requirements:
- high health
- visible large placeholder model
- intro movement
- multiple attack phases
- phase changes based on health percentage
- emits boss_defeated signal
- gives large score reward
- triggers mission complete when defeated

Boss phases:
Phase 1: simple forward fire
Phase 2: spread shots
Phase 3: faster fire + side turrets
Phase 4: damaged desperation pattern

Acceptance criteria:
- Boss appears at end of wave.
- Boss has multiple phases.
- Boss can be damaged and defeated.
- Mission complete triggers on boss defeat.
- HUD can display boss health if simple.
```

Phase 14 - Game States and Menus

```
Implement Phase 14: game states and menus.

Create:
- scenes/ui/TitleScreen.tscn
- scenes/ui/PauseMenu.tscn
- scenes/ui/GameOverScreen.tscn
- scenes/ui/MissionCompleteScreen.tscn
- scripts/ui/title_screen.gd
- scripts/ui/pause_menu.gd
- scripts/ui/game_over_screen.gd
- scripts/ui/mission_complete_screen.gd

Game states:
- TITLE
- PLAYING
- PAUSED
- GAME_OVER
- MISSION_COMPLETE

Requirements:
- Start game from title screen
- Pause/unpause during gameplay
- Restart after game over
- Continue to next mission after mission complete, even if only one mission exists for now
- No gameplay movement while paused or in menus

Acceptance criteria:
- Game starts from title screen.
- Pause menu works.
- Game over screen works.
- Mission complete screen works.
- Restart returns to a playable state.
```

Phase 15 - Effects Pass

```
Implement Phase 15: effects pass.

Create:
- scenes/effects/Explosion.tscn
- scenes/effects/SmokeTrail.tscn
- scenes/effects/HitSpark.tscn
- scenes/effects/WaterSplash.tscn
- scripts/effects/explosion.gd
- scripts/effects/effects_manager.gd
- scripts/effects/screen_shake.gd
```

```
Effects:
- enemy death explosion
- bullet hit spark
- boss damage smoke
- aircraft water crash splash
- small screen shake on big explosions
- no excessive visual clutter

Use placeholders if no final particles/materials exist.

Acceptance criteria:
- Enemy death spawns explosion.
- Bullet hit spawns hit spark.
- Boss damage can show smoke.
- Effects clean themselves up or return to pool.
- Screen shake does not make gameplay unreadable.
```

Phase 16 - Audio Hooks

```
Implement Phase 16: audio manager and audio hooks.

Create:
- scripts/systems/audio_manager.gd
- placeholder references for:
  - player_fire
  - enemy_destroyed
  - player_hit
  - powerup_collect
  - boss_appear
  - mission_complete
  - game_over
  - menu_select
  - stage_music

Requirements:
- Use AudioStreamPlayer or AudioStreamPlayer3D appropriately.
- Do not require final audio assets.
- If placeholder audio files are missing, fail gracefully without errors.
- Add documentation explaining where to place audio files.

Acceptance criteria:
- Gameplay calls audio hooks.
- Missing audio does not crash the game.
- Volume can be controlled globally later.
- README explains audio asset placement.
```

Phase 17 - First Complete Stage

```
Implement Phase 17: first complete stage.

Create Stage 1: Open Pacific.

Stage content:
- scrolling ocean
- clouds
- at least 3 fighter waves
- at least 1 bomber wave
- at least 1 power-up drop
- enemy bullets
- boss encounter
- mission complete screen

Update:
- data/stages/stage_01.json
- data/waves/stage_01_waves.json
- docs/game_design.md

Stage duration target:
2 to 4 minutes for the prototype.

Acceptance criteria:
- Player can start from title screen.
- Player can play through Stage 1.
- Enemies spawn in waves.
- Player can die and get game over.
- Player can defeat boss and get mission complete.
```

- Score/lives/HUD work throughout.

Phase 18 - Ultra-Realistic Visual Direction Pass

```
Implement Phase 18: visual realism preparation.

Do not add heavy final assets yet unless they already exist.

Create documentation and placeholder material system for:
- realistic aircraft models
- ocean material
- cloud layers
- explosion particles
- smoke trails
- tracer bullets
- aircraft shadows
- water splashes
- ship wakes
- island chunks

Update:
- docs/art_direction.md
- docs/technical_plan.md
- assets/materials/README.md
- assets/models/README.md
- assets/textures/README.md

If useful, create simple placeholder materials:
- aircraft metal
- ocean blue
- smoke dark
- tracer glow
- explosion fire

Acceptance criteria:
- Project has clear asset replacement pipeline.
- Placeholder materials exist or are documented.
- Art direction explains top-down realistic style.
- Gameplay remains readable.
```

Phase 19 - Performance Optimization

```
Implement Phase 19: performance and cleanup pass.

Review the project for:
- excessive instantiation
- bullets not cleaning up
- effects not cleaning up
- unused scripts
- giant scripts that should be split
- repeated code
- weak signal connections
- hard-coded paths that should be exported variables
- missing error handling for JSON loading

Improve:
- object pooling
- cleanup logic
- JSON load validation
- scene references
- comments where helpful
- docs/known_issues.md

Do not change gameplay feel unless needed for a bug fix.

Acceptance criteria:
- Gameplay still works.
- No obvious memory leaks from bullets/effects.
- JSON loading has graceful error messages.
- docs/known_issues.md is updated.
```

Phase 20 - Export Preparation

```
Implement Phase 20: export preparation.
```

```
Create documentation for:
- how to export Windows build
- how to export Linux build
- how to export web build later
- how to export Android later
- how to replace placeholder assets
- how to test controls
- how to package the game

Create:
- docs/export_guide.md
- docs/testing_checklist.md
- docs/release_checklist.md

Add:
- credits template
- license note
- asset license tracking file

Do not assume final commercial release.
This is a prototype release package.

Acceptance criteria:
- Developer can follow docs/export_guide.md.
- testing_checklist.md covers core gameplay.
- release_checklist.md covers final manual checks.
```

Part 4 - Technical Blueprint

Architecture

The project should behave like a 2D arcade shooter but render as a 3D top-down scene. This gives modern visuals without simulator complexity.

Main scene tree

```
Main
  WorldRoot
    Ocean
    IslandSpawner
    CloudLayer
    ShipLayer
    Player
    Enemies
    PlayerBullets
    EnemyBullets
    Effects
  EnemySpawner
  StageManager
  GameState
  AudioManager
  Camera3D
  DirectionalLight3D
  WorldEnvironment
  HUD
```

Core script responsibilities

Script	Responsibility
main.gd	scene initialization, root references, high-level coordination
game_state.gd	score, lives, bombs, stage state, game over/mission complete
player.gd	arcade movement, bounds, damage state, signals
player_weapon.gd	fire cooldowns, weapon levels, bullet spawning
bullet_base.gd	projectile movement, damage, cleanup, hit handling
enemy_base.gd	health, score value, movement pattern, damage/death
enemy_spawner.gd	JSON wave loading and timed spawning
movement_patterns.gd	straight, diagonal, sine, boss intro, dive patterns
hud.gd	score/lives/bombs/stage/boss health display
audio_manager.gd	sound hooks and graceful missing-asset handling

Gameplay Principles

- Movement must be responsive and arcade-like. Do not implement realistic lift, drag, stalls, fuel, or long aircraft turn radius.
- The visual model can be large, but the damage hitbox should be small and fair.
- Bullets must remain readable even with realistic graphics. Use tracer glow, contrast, and consistent colour language.
- Stages should be data-driven. Do not hard-code entire levels into scripts.

Data Examples

enemy_types.json

```
{
  "fighter": {
    "scene": "res://scenes/enemies/EnemyFighter.tscn",
    "health": 2,
    "score_value": 100,
    "speed": 6.0,
    "fire_interval": 2.0,
    "drop_chance": 0.05
  },
  "heavy_fighter": {
    "scene": "res://scenes/enemies/EnemyFighter.tscn",
    "health": 5,
    "score_value": 250,
    "speed": 4.0,
    "fire_interval": 1.5,
    "drop_chance": 0.08
  },
  "bomber": {
    "scene": "res://scenes/enemies/EnemyBomber.tscn",
    "health": 12,
    "score_value": 800,
    "speed": 2.5,
    "fire_interval": 1.2,
    "drop_chance": 0.25
  },
  "boss_bomber": {
    "scene": "res://scenes/enemies/BossBomber.tscn",
    "health": 150,
    "score_value": 5000,
    "speed": 1.2,
    "fire_interval": 0.9,
    "drop_chance": 0.0
  }
}
```

stage_01_waves.json

```
[
  {
    "time": 1.0,
    "enemy_type": "fighter",
    "spawn_x": -4.0,
    "spawn_z": -18.0,
    "movement": "diagonal_right"
  },
  {
    "time": 1.3,
    "enemy_type": "fighter",
    "spawn_x": 4.0,
    "spawn_z": -18.0,
    "movement": "diagonal_left"
  },
  {
    "time": 3.0,
    "enemy_type": "fighter",
    "spawn_x": 0.0,
    "spawn_z": -18.0,
    "movement": "straight_down"
  },
  {
    "time": 5.0,
    "enemy_type": "fighter",
    "spawn_x": -2.5,
    "spawn_z": -18.0,
    "movement": "sine_wave"
  },
  {
    "time": 5.2,
    "enemy_type": "fighter",
    "spawn_x": 2.5,
    "spawn_z": -18.0,
    "movement": "sine_wave"
  },
  {
    "time": 8.0,
    "enemy_type": "bomber",
    "spawn_x": 0.0,
```

```
    "spawn_z": -20.0,
    "movement": "slow_center"
  },
  {
    "time": 13.0,
    "enemy_type": "heavy_fighter",
    "spawn_x": -5.0,
    "spawn_z": -18.0,
    "movement": "diagonal_right"
  },
  {
    "time": 13.0,
    "enemy_type": "heavy_fighter",
    "spawn_x": 5.0,
    "spawn_z": -18.0,
    "movement": "diagonal_left"
  },
  {
    "time": 20.0,
    "enemy_type": "boss_bomber",
    "spawn_x": 0.0,
    "spawn_z": -24.0,
    "movement": "boss_intro"
  }
]
```

weapon_types.json

```
{
  "level_1": {
    "name": "Dual machine guns",
    "bullet_count": 2,
    "fire_cooldown": 0.18,
    "spread_degrees": 0,
    "damage": 1
  },
  "level_2": {
    "name": "Fast dual guns",
    "bullet_count": 2,
    "fire_cooldown": 0.13,
    "spread_degrees": 4,
    "damage": 1
  },
  "level_3": {
    "name": "Triple shot",
    "bullet_count": 3,
    "fire_cooldown": 0.15,
    "spread_degrees": 12,
    "damage": 1
  },
  "level_4": {
    "name": "Wide spread shot",
    "bullet_count": 5,
    "fire_cooldown": 0.18,
    "spread_degrees": 22,
    "damage": 1
  },
  "rocket": {
    "name": "Rocket attack",
    "bullet_count": 1,
    "fire_cooldown": 0.8,
    "spread_degrees": 0,
    "damage": 8
  }
}
```

powerup_types.json

```
{
  "weapon_upgrade": {
    "label": "Weapon Upgrade",
    "effect": "increase_weapon_level",
    "duration": 0
  },
  "repair": {
    "label": "Repair",
    "effect": "restore_life_or_health",
    "duration": 0
  },
```

```
  "bomb": {
    "label": "Bomb",
    "effect": "add_bomb",
    "duration": 0
  },
  "score_bonus": {
    "label": "Score Bonus",
    "effect": "add_score",
    "amount": 1000,
    "duration": 0
  },
  "rapid_fire_temporary": {
    "label": "Rapid Fire",
    "effect": "temporary_fire_rate_boost",
    "duration": 8
  }
}
```

stage_01.json

```
{
  "id": "stage_01",
  "name": "Open Pacific",
  "wave_file": "res://data/waves/stage_01_waves.json",
  "environment": "open_pacific_day",
  "music": "stage_01_music",
  "boss": "boss_bomber",
  "target_duration_seconds": 180
}
```

Part 5 - Testing and Debugging

Per-Phase Testing Checklist

```
After every phase, check:

1. Godot opens the project.
2. Main.tscn loads.
3. Press Play.
4. No script parse errors.
5. No missing scene reference errors.
6. Player still appears.
7. Player still moves.
8. Player can still shoot if shooting is implemented.
9. Enemies still spawn if spawner is implemented.
10. Bullets still collide correctly.
11. Score/lives/HUD still update.
12. Pause/game over still works if implemented.
13. No console spam.
14. No huge performance drop.
15. docs/known_issues.md is updated.
```

Debugging Prompts

When Godot throws an error, copy the exact error into one of these prompts. Keep fixes narrow and targeted.

General Error

```
The Godot project has this error:

[paste exact error]

Please fix the smallest possible cause.
Do not rewrite unrelated systems.
Explain which file/path caused it.
Keep the project runnable.
Update docs/known_issues.md if the issue reveals an unfinished system.
```

Scene Reference Error

```
Godot reports a missing or invalid scene/script reference:

[paste exact error]

Inspect the related .tscn and .gd files.
Fix resource paths, node paths, exported scene references, or preload paths as needed.
Do not change gameplay behaviour unless required.
```

Collision Not Working

```
Collision is not working correctly:

Observed behaviour:
[describe what happens]

Expected behaviour:
[describe expected result]

Check:
- Area3D setup
- CollisionShape3D
- collision layers/masks
- signal connections
- bullet hit logic
- enemy take_damage()
- player take_damage()

Fix the minimum necessary code and document the cause.
```

Player Movement Broken

```
Player movement is broken:

Observed:
[describe]

Expected:
The player should move on the X/Z plane using move_left, move_right, move_up, move_down and stay
    inside playfield bounds.

Check player.gd, input actions, Main.tscn node references, and project.godot input map.
Fix only the movement/input issue.
```

Json Wave Loading Broken

```
Enemy wave loading is broken:

Observed:
[describe]

Expected:
EnemySpawner should load data/waves/stage_01_waves.json and spawn enemies at the specified
    times.

Check:
- JSON file path
- JSON syntax
- FileAccess usage
- parsed data validation
- enemy type mapping
- scene preload/instantiate logic

Fix the loader and add graceful error messages.
```

Performance Issue

```
Performance drops when many bullets or effects are active.

Please inspect:
- bullet creation/destruction
- enemy bullet creation/destruction
- effect spawning
- cleanup logic
- object pooling

Implement the smallest safe optimization.
Do not change gameplay feel.
```

Pull Request Template

```
## Summary

Describe what this PR changes.

## Phase

Which development phase does this PR implement?

## Gameplay Impact

- [ ] No gameplay change
- [ ] Player movement
- [ ] Shooting
- [ ] Enemies
- [ ] Collision
- [ ] HUD
- [ ] Stage/waves
- [ ] Effects/audio
- [ ] Menus/game states
- [ ] Performance/export

## Test Checklist

- [ ] Project opens in Godot
- [ ] Main scene runs
```

```
- [ ] No script parse errors
- [ ] No missing critical scene references
- [ ] Feature works in a basic manual test
- [ ] Existing features still work
- [ ] README/docs updated if needed

## Known Issues

List known unfinished items or bugs.

## Screenshots / Notes

Add screenshots or gameplay notes if relevant.
```

Release Checklist

- Project opens in the target Godot version.
- Main scene runs without parse errors.
- One full stage is playable from title to mission complete.
- Game over and restart work.
- Score, lives, bombs, and stage UI update correctly.
- Bullets, effects, and enemies clean up or return to pools.
- Placeholder assets are clearly marked.
- No copied copyrighted assets are included.
- Export guide and testing checklist are updated.

Part 6 - Visual Art Direction

Art Direction

The game should look like a realistic top-down Pacific air combat scene while playing like a fast arcade shooter. Readability is more important than pure realism.

Realistic aircraft

- Top-down silhouettes must be distinct.
- Use weathered metal, propeller blur, smoke, damage states, and aircraft shadows.
- Use simplified versions at small screen size to preserve readability.

Ocean and islands

- Use animated water material, subtle normal maps, foam patches, wake trails, oil slicks, splash effects, and island chunks.
- Avoid visual noise under bullets and enemies. Gameplay clarity wins.

Effects

- Aircraft explosions should include fireball, smoke, debris, shockwave, and water splash when appropriate.
- Screen shake should be light and short.
- Effects must clean themselves up or use pooling.

Visual Asset Prompts

```
Player aircraft asset spec:
Create a photorealistic top-down WWII-inspired twin-engine fighter aircraft for a vertical
    scrolling shooter.
Requirements: viewed from above, clean readable silhouette, original design, metallic body with
    subtle weathering, visible wings/cockpit/engines/propeller blur, suitable for orthographic
    top-down gameplay.

Enemy fighter asset spec:
Create a photorealistic top-down WWII-inspired enemy fighter aircraft for an arcade vertical
    shooter.
Requirements: viewed from above, clean silhouette, darker enemy colour scheme, realistic metal
    and paint wear, propeller blur, readable at small size, original design.

Ocean material spec:
Create a realistic Pacific ocean material for a top-down vertical scrolling aerial shooter.
Requirements: deep blue water, subtle wave normal map, sunlight reflection, foam patches,
    readable under aircraft and bullets, not too visually noisy, works from orthographic
    top-down camera.

Explosion effect spec:
Create a cinematic aircraft explosion effect for a top-down realistic arcade shooter.
Requirements: fireball, dark smoke, sparks, metal debris, brief shockwave, readable from
    top-down camera, not too large to obscure gameplay.
```

Stage 1 Design Prompt

```
Design and implement Stage 1: Open Pacific.

Theme:
Calm Pacific ocean, small islands, scattered clouds, early enemy patrols.

Gameplay duration:
2 to 4 minutes for prototype.
```

```
Stage flow:
1. Opening calm ocean with two small fighter waves.
2. Diagonal fighter wave from left and right.
3. First bomber enters center with escort fighters.
4. Power-up drop after bomber.
5. Enemy bullets increase slightly.
6. Mini-wave of sine-wave fighters.
7. Boss warning.
8. BossBomber appears.
9. Boss has 3 phases.
10. Mission complete screen after boss defeat.

Use data-driven JSON waves.
Do not hard-code the full stage in scripts.
Update docs/game_design.md with Stage 1 notes.
```